WNDRVR
SUPPORT NETWORK

VXWORKS 7 POWERPC ARCHITECTURE GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 PowerPC Architecture Guide, SR0650

10 September 2020

1. POWERPC PROCESSOR SUPPORT FOR VXWORKS 7

1. PowerPC Processor Support

VxWorks® 7 includes support for PowerPC 32-bit and 64-bit processors.

The following table provides a basic breakdown of the processor support available in this release of VxWorks 7. For more information related to supported system-on-a-chip (SoC) designs, see Support for NXP SoCs on page 1.

CPU Type	Description	PowerPC 32-Bit Support?	PowerPC 64-Bit Support?
PPC603	Includes MPC603 processor cores.	Yes	No
PPC604	Includes MPC750 processor cores.	Yes	No
PPCE500MC	Includes e500mc and e5500 processor cores.	Yes	Yes (for e5500 only)
PPCE500V2	Includes e500v2 processor cores.	Yes	No
PPCE6500	Includes e6500 processor cores.	Yes	Yes

Both the e5500 and PPC750 are supported cores. However, this release does not include a CPU option specific to these cores.

- For e5500, support is provided by the PPCE500MC option. (For more information, see reference document for the fsl_p3p4p5 BSP.)
- For PPC750, support is provided by the PPC604 option. (There is no reference BSP shipped for this processor type in this release.)

Note: Support for additional processor core types may be added periodically. For the most recent support information, see the Wind River support website.

2. Support for NXP SoCs

When developing your VxWorks for PowerPC system for a specific NXP system-on-a-chip (SoC) design, you should understand how VxWorks processor support relates to your SoC.

The following core types, defined by NXP, are supported in this release of VxWorks:

- PPC603
- e500v2
- e500mc
- e5500
- e6500

NXP uses these cores as part of their overall system on a chip (SoC) designs.

Note: PPC750 (as supported by the PPC604 CPU option) is also supported but is not relevant to the SoC discussion.

Note: While these SOCs are the property of NXP Semiconductors N.V., they were originally developed by Freescale Semiconductor, Inc. Wind River often still refers to these PowerPC SOCs as Freescale devices.

The breakdown of what core is in what NXP SoC is as follows:

PPC603	e500v2	e500mc	e5500	e6500
MPC8260	MPC8548	P4080	P5020	T4240
MPC8265	MPC8572	P3041	P5040	T2080
	P2020			

Note: This is a representative list only. Additional SoCs can be supported.

VxWorks provides 32-bit support for the following cores: PPC603, e500v2, and e500mc. VxWorks 64-bit support is provides for e5500 and e6500 cores.

2. POWERPC ARCHITECTURE INFORMATION

When working with a PowerPC-based target system, you should be aware of certain characteristics of the architecture.

Byte Order

The byte order used by VxWorks for the PowerPC family is big-endian.

Divide-by-Zero Handling

Integer division by zero produces undefined results. Exception generation and handling are not provided by the compiler or run time.

Floating-point exceptions are disabled by default during task initialization, causing zero-divide conditions to be ignored. On processors with hardware floating-point (for example, e500mc, e6500, or MPC603), individual tasks can modify their machine state register (MSR) and the floating-point status and control register (FPSCR) in order to generate exceptions. Likewise, for the e500v2, the SPEFSCR and MSR must be modified to generate exceptions. On processors without hardware floating-point, neither the software floating-point library nor the compiler provide support for simulating a floating-point exception.

Hardware Breakpoints

The PowerPC family supports hardware breakpoints. However, the access type of hardware breakpoints allowed depends upon the specific processor.

For information on the number of hardware breakpoints (a hardware limitation), address alignment constraints, and access types, see the hardware reference information for your specific processor.

For more information, see the reference entry for bh( ).

Stack Frame Alignment

The stack frame alignment for all PowerPC CPU families is 16 bytes.

In general, Wind River recommends that you recompile your code and that you do not reuse objects compiled for a different environment, including an older version of VxWorks.

Register Usage

The PowerPC conventions regarding register usage, stack frame formats, parameter passing between functions, and other factors involving code inter-operability, are defined by the Application Binary Interface (ABI) and the Embedded Application Binary Interface (EABI) protocols. The VxWorks implementation for PowerPC follows these protocols.

The following table shows 32-bit PowerPC register usage in VxWorks (note that only CPUs with hardware floating-point support have fpr0-31).

Table 1. 32-Bit PowerPC Register Usage

Register Name	Usage
gpr0	Volatile register that may be modified during function linkage.
gpr1	Stack frame pointer, always valid.
gpr2	Address thread local storage (TLS) relative to this thread pointer.
gpr3	Volatile register used for parameter passing and return values.
gpr4 - gpr10	Volatile registers used for parameter passing.
gpr11 - gpr12	Volatile registers that may be modified during function linkage.
gpr13	Small data area pointer register (**_SDA_BASE_**).
gpr14 - gpr30	Non-volatile registers used for local variables.
gpr31	Used for local variables or environment pointers.
sprg0 - sprg3	Special purpose registers; used by VxWorks.
sprg4 - sprg7	Book E (PPCE500V2/PPCE500MC/PPCE6500 and PowerPC 4xx) special purpose registers; used by VxWorks.
usprg0	Book E special purpose register; not used by VxWorks.
fpr0	Volatile floating-point register.
fpr1	Volatile floating-point register used for parameter passing and return values.
fpr2 - fpr8	Volatile floating-point registers used for parameter and results passing.
fpr9 - fpr13	Volatile floating-point registers.
fpr14 - fpr31	Non-volatile floating-point registers used for local variables.

The following table shows 64-bit PowerPC register usage in VxWorks. PowerPC 64-bit follows the ELF V2 ABI protocol.

Table 2. 64-Bit PowerPC Register Usage

Register Name	Usage
gpr0	Volatile register. Optional in function linkage. Used in function prologues.
gpr1	Stack frame pointer, non-volatile.
gpr2	TOC pointer. This register is non-volatile with respect to calls between functions in the same compiled unit of code.
gpr3 - gpr10	Volatile registers used for parameter and return values.
gpr11	Volatile register.
gpr12	Volatile register. This is the function entry address at the global entry point.
gpr13	Address thread local storage (TLS) relative to this thread pointer.
gpr14 - gpr31	Non-volatile register used for local variables.
LR	Volatile link register.

Physical Addressing Support

The following table provides information on physical addressing for supported PowerPC architectures:

Note: Support for physical addressing space greater than 32 bits varies by architecture.

Architecture Variant	Virtual Addressing Size	Size of PHYS_ADDR	Supported PHYS_ADDR Bits
PPCE500V2	32-bit	64-bit	32-bit 36-bit
PPCE500MC (32-bit)	32-bit	64-bit	32-bit 36-bit
PPCE500MC (64-bit)	64-bit	64-bit	36-bit
PPCE6500 (32-bit)	32-bit	64-bit	32-bit 40-bit
PPCE6500 (64-bit)	64-bit	64-bit	40-bit
PPC603	32-bit	64-bit	32-bit

Note: The PPCE500MC CPU option supports 64-bit for e5500 cores only. For more information, see Support for NXP SoCs on page 1.

Additional Information

Comprehensive information regarding PowerPC hardware behavior and programming is beyond the scope of this document. IBM and NXP Semiconductors provide several hardware and programming manuals for the PowerPC processor on their websites:

- http://www.nxp.com/
- http://www.ibm.com/
- http://www.openpowerfoundation.org/

Wind River recommends that you consult the hardware documentation for your processor or processor family as necessary during BSP development. The references provided below are current at the time of writing. Should you decide to use these documents, you may to contact the manufacturer for the most current version.

- The PowerPC Architecture: A Specification for a New Family of RISC Processors, Morgan-Kaufmann, 1994, ISBN 1-55860-316-6.

Available from NXP Semiconductors/Freescale:

- Programming Environments Manual for 32-bit Implementations of the PowerPC Architecture, Order #MPCFPE32B, 9/2005.
- EREF 2.0: A Programmer's Reference Manual for Freescale Power Architecture Processors
- E500, E500MC, E6500, E5500 Core Reference Manuals

3. VXWORKS LIMITATIONS FOR POWERPC

Not all features of VxWorks are applicable to or supported for PowerPC-based systems. You should be aware of these limitations before beginning your development.

> Note: Your release may include additional limitations or usage caveats. For more information, see the VxWorks 7 Release Notes.

Code Model Considerations for PowerPC 64

The 32-bit PowerPC ELF ABI specification defines a global offset table (GOT) section that is used to hold addresses for position-independent code. The specification also defines a small data section to hold small global data variables. The same register is sometimes used to address both the GOT and the small data section. However, the 64-bit PowerPC ELFv2 ABI defines a table of contents (TOC) section. The TOC combines the functions of the GOT and the small data section. The TOC region commonly includes data items within the .got, .toc, .sdata, and .sbss sections. That is, the TOC section contains a conventional ELF GOT, and can optionally contain a small data area. The GOT and the small data area can be intermingled in the TOC section.

The TOC section is accessed using the dedicated TOC pointer register, r2. The value of the TOC pointer register is called the TOC base. The TOC base is typically the first address in the TOC plus 0x8000.

A relocatable object file must have a single TOC section and a single TOC base. However, when the link editor combines relocatable object files to form a single executable or shared object, it can create multiple TOC sections. In VxWorks, kernel images are built by linking multiple objects. For performance reasons, VxWorks allows only a single TOC section for the kernel image.

Your compiler can provide different code models depending on the expected size of the TOC and the size of the entire executable or shared library.

> Note: Shared libraries are not available in this release of VxWorks for PowerPC 64. For more information, see your product release notes.

The three available code models are:

Small code model	The TOC is accessed using 16-bit offsets from the TOC pointer. This limits the size of a single TOC to 64 KB. Position-independent code uses GOT-indirect addressing to access other objects in the binary.
Large code model	The TOC is accessed using 32-bit offsets from the TOC pointer, except for .sdata and .sbss, which are accessed using 16-bit offsets from the TOC pointer. This allows a TOC of at least 2 GB. Position-independent code uses GOT-indirect addressing to access other objects in the binary.
Medium code model	Like the large code model, the TOC is accessed using 32-bit offsets from the TOC pointer, except for .sdata and .sbss, which are accessed using 16-bit offsets. In addition, accesses to module-local code and data objects use TOC pointer relative addressing with 32-bit offsets. Using TOC pointer relative addressing removes a level of indirection, resulting in faster access and a smaller GOT. However, it limits the size of the entire binary to between

	2 GB and 4 GB, depending on the placement of the TOC base.

The medium code model provides a sufficiently large TOC to provide all data addressing needs of a module within a single TOC. The compiler can generate two-instruction medium code model references (or, if selected, short displacement one-instruction references) for all data items that are in the TOC for the object file being compiled. Such references are relative to the TOC pointer register, r2.

VxWorks for PowerPC 64 includes the following restrictions with respect to code models:

- For kernel images, downloadable kernel modules (DKMs), and real-time process (RTP) applications, there is only a single TOC for each image of them.
- The large code model is not currently supported.
- Shared libraries for RTPs are not supported as they are not supported by the compiler.
- For kernel images can only use the medium code model currently.
- For DKM and RTP images, you can select either the small or medium code model. Choosing the small code model, when size permits, can improve performance in some cases.

Light Power Management Support

Light power management support (INCLUDE_CPU_LIGHT_PWR_MGR) is only available for the 32-bit PowerPC architecture in this release. This support is not available for 64-bit PowerPC systems.

Restrictions on tt( )

The tt( ) function makes assumptions about the standard prolog for functions. If functions are written in assembly language, or in another language that generates a different prolog, the tt( ) function can generate unexpected results.

The tt( ) function does not report the parameters to C functions as it cannot determine these from the code generated by the compiler.

Restrictions on vxTas( )

The vxTas( ) function provides a C-callable interface to a test-and-set instruction. Due to hardware limitations, the operand of vxTas( ) must be either cached-copyback or uncached (that is, it cannot be cached-writethrough).

Note:	Semiconductor vendors indicate that future PowerPC processors are likely to require the operand of vxTas( ) to be a cached-copyback address.

Shared Library Support for RTPs

Previously, shared library support for real-time processes (RTPs) was only available for the 32-bit PowerPC architecture. This support was not available for PowerPC 64-bit.

With the switch to a non-Wind River specific version of the GNU compiler, shared libraries are not supported for 32-bit in this particular release. Therefore, shared libraries are not supported for 32-bit and 64-bit PowerPC architectures in this particular release.

Small Data Area for PowerPC 32

The GNU compiler supports the small data area (SDA) feature for 32-bit PowerPC systems. This release of VxWorks for PowerPC supports the small data area feature in the kernel (but not for downloadable kernel modules) as well as in RTPs. Kernel code must not

modify gpr13 other than in connection with context switching, because this registers is used to point to the active task SDA segment **.sda**.

Note: The **SDA_DISABLE** makefile variable is supplied for the purposes of generating a downloadable kernel module (DKM). When generating a DKM, set **SDA_DISABLE** to **TRUE** to instruct the compiler to not use SDA for object module generation. However, you should note that this setting does not disable SDA in the kernel environment and you must still be sure that your code does not modify the reserved register (gpr13).

Only a select body of VxWorks source code is compiled with SDA support. This limitation is used to prevent SDA section overflow. The **usrAppInit.c** configlette is not built with SDA support to allow customer initialization code to reference global variables in their applications without encountering a linker error. By default, user applications are built without SDA support (again, to prevent SDA section overflow). If your application code must be compiled with SDA support, you can use the -G8 -msdata=sysv compiler option for the GNU compiler. However, be sure to exercise caution with this option to prevent section overflow.

Thread Local Storage (TLS)

For 32-bit and 64-bit PowerPC architectures, VxWorks tasks can store global variables on a per-task basis in the TLS area. On 32-bit architectures, the TLS for each VxWorks task is accessed relative to the address in **gpr2**. On 64-bit architectures, the task's TLS is accessed relative to the address in **gpr13**.

Software Floating-Point Support

Software floating-point support is available for both the 32-bit and 64-bit PowerPC architecture in this release.

WNDRVR

4. VXWORKS SOURCE BUILD CONFIGURATION CONSIDERATIONS FOR POWERPC

VxWorks 7 includes certain VxWorks source build (VSB) configuration options specific to PowerPC systems.

PowerPC 32 and 64

Consider the following VSB configuration options when configuring your PowerPC system for kernel applications:

PPC_CACHE_MODE_IS_CONSISTENT

Enabling this option removes cache management related to enabling and disabling the MMU. This can improve performance and reduce overhead in your system. For more information, see the help entry for this option.

PPC_DRV

This option provides the PowerPC common drivers.

PPC_E500_BRANCH_PREDICTION

For PowerPC E500-based systems, this option is used to enable branch prediction. For other PowerPC architectures, it is ignored.

PPC_PPCMATH and CH_PPC_PPCMATH

When enabled, PPC_PPCMATH provides the PowerPC math libraries. If the math libraries are enabled, CH_PPC_PPCMATH allows you to set the version of the math libraries to be used.

Consider the following VSB option when configuring your PowerPC system for user side applications:

PPC_USER

This option enables the user-side libraries in your PowerPC system.

Additional Options for PowerPC 64

Consider the following additional configuration options specific to PowerPC 64 when configuring your VSB for kernel applications:

PPC64_CODE_MODEL_CHOICE

In this release, the only allowed value for this option is PPC64_MEDIUM_CODE_MODEL. This value is set by default for VSBs based on PowerPC 64 hardware.

> Note: The PPC64_SMALL_CODE_MODEL and PPC64_LARGE_CODE_MODEL values are not supported for kernel applications at this time.

Consider the following additional configuration options specific to PowerPC 64 when configuring your VSB for user side applications:

PPC64_USER_CODE_MODEL_CHOICE

The allowed values for this release are:

- PPC64_USER_SMALL_CODE_MODEL
- PPC64_USER_MEDIUM_CODE_MODEL (default)

Note: At this time, the PPC64_USER_LARGE_CODE_MODEL option is not supported for user side applications.

If your real-time process (RTP) application requires less than 64 KB of TOC space, you can select the PPC64_USER_SMALL_CODE_MODEL option. This option can improve performance in systems requiring less than 64 KB of TOC space. If you build your downloadable kernel module (DKM) or RTP applications using Workbench 4, you can select the proper build specification according to the needs of your application.

Note: If you attempt to build using the small code model for an application that requires greater than 64 KB of TOC space, your build will fail and you will be prompted to change to the medium code model, PPC64_USER_MEDIUM_CODE_MODEL.

5. ASSEMBLY LANGUAGE MACROS FOR POWERPC

There are several macros available to you when programming in assembly language for PowerPC 32-bit and 64-bit targets.

HI and HADJ Macros

The **HI** and **HIADJ** macros are used in PowerPC assembly code to facilitate the loading of immediate operands larger than 16 bits. These macros are available for use with both 32-bit and 64-bit PowerPC platforms. The **HI(***symbol***)** macro is the simple high-order 16 bits of the value *x*. The **HIADJ(***symbol***)** macro is the high-order 16 bits adjusted by the MSB (most significant bit) of the low-order 16 bits of value *symbol*. That is, if the MSB is set, **HIADJ(***symbol***)** truncates the lower 16 bits and adjusts the resulting value by adding 1 to the upper 16 bits.

The **HIADJ(***symbol***)** macro must be used whenever the low-order 16 bits are used in an instruction that interprets them as a signed quantity (for instance, **addi** or **lwz**). If the low-order bits are used in an instruction that interprets them as an unsigned quantity (for instance, **ori**), the proper macro **HI**, not **HIADJ**, should be used.

For example, **addi** uses a signed quantity, so **HIADJ** is the proper macro:

```
lis   rx, HIADJ(VALUE)
addi  rx, rx, LO(VALUE)
```

However, **ori** uses an unsigned quantity, so **HI** is the proper macro:

```
lis   rx, HI(VALUE)
ori   rx, rx, LO(VALUE)
```

Available PowerPC Assembly Macros

To obtain the multiple 16-bit segments of a 64-bit symbol in assembly language source code, use the following macros:

Note: To use these macros, include the following header in your source code: **<arch/ppc/archPpc.h>**

Macro	Description	Compiler/ABI Notation
HITA(*symbol***)**	Highest (most-significant 16 bits) adjusted part of the offset.	symbol@highesta
HIT(*symbol***)**	Highest (most-significant 16 bits) part of the offset.	symbol@highest
HIRA(*symbol***)**	Higher (next significant 16 bits) adjusted part of the offset.	symbol@highera
HIR(*symbol***)**	Higher (next significant 16 bits) part of the offset.	symbol@higher
HIADJ(*symbol***)** or **HIGHA(***symbol***)**	High (next significant 16 bits) adjusted part of the offset.	symbol@higha
HIGH(*symbol***)** or **HI(***symbol***)**	High (next significant 16 bits) part of the offset.	symbol@high

Macro	Description	Compiler/ABI Notation
LO(*symbol***)**	Low part of the offset.	symbol@h

Note: The **HIADJ**, **HI**, and **LO** macros are also available for use with 32-bit PowerPC platforms.

To load a symbol, or content addressed by a symbol, into a register in assembly language source code, use the following macros:

Macro	Description
SET_HIWORD(*reg, symbol***)**	Loads the high 32 bits of the symbol value into the high 32 bits of the register. As a result, the low 32 bits of the register are overwritten. Note: This macro is also available for PowerPC 32-bit platforms. However, it is just a placeholder for source code compatibility with 64-bit PowerPC. It does not affect the register content because a 32-bit PowerPC register has only 32 bits.
SET_HIADJ(*reg, symbol***)**	Loads the high 48 bits of the symbol value into the high 48 bits of the register. As a result, the low 16 bits of the register are overwritten. Note: This macro is also available for use with PowerPC 32-bit platforms. However, it only loads the high 16 bits of the symbol value into the high 16 bits of the register because PowerPC 32-bit systems only have 32 bits available.

With the above two macros, you can load the high adjusted part of a symbol into a register. You can then use the remaining low 16 bits of the symbol as an offset (with regard to the value in the register) to do load and store operations. For example, the following macros are created by the above macros:

LOADPTR(*reg, symbol***)**

Loads the full 64 bits of the symbol into the 64 bit register.

This macro is created as follows:

```
#define LOADPTR(reg, symbol) \
        SET_HIADJ(reg, symbol); addi reg,reg,LO(symbol)
```

Note: This macro is also available for use with PowerPC 32-bit systems. For those systems, it loads 32 bits into the register.

LOADVAR(*reg, symbol***)** and **STOREVAR(***v, sc, symbol***)**

Loads the 32-bit value at the address represented by the symbol into the register. This is mostly used to load variable values defined as the **int** or **unsigned int** type in a C source program from an assembly language program.

This macro is created as follows:

```
#define LOADVAR(reg, symbol) \
        SET_HIADJ(reg, symbol); lwz reg, LO(symbol)(reg)
#define STOREVAR(v, sc, symbol) \
        SET_HIADJ(sc, symbol); stw v, LO(symbol)(sc)
```

LOADVAR_LP(*reg, symbol***)** and **STOREVAR_LP(***v, sc, symbol***)**

Loads the 32-bit (for PowerPC 32) or 64-bit (for PowerPC 64) value at the address represented by the symbol into the register. This is mostly used to load variable values defined as the **long** or **unsigned long** type in a C source program from an assembly language program.

This macro is created as follows:

```
#define LOADVAR_LP(reg, symbol) \
        SET_HIADJ(reg, symbol); LDLP reg, LO(symbol)(reg)
#define STOREVAR_LP(v, sc, symbol) \
        SET_HIADJ(sc, symbol); STLP v, LO(symbol)(sc)
```

To load a global variable (content addressed by a global symbol) into a register through small data area (SDA) (for PowerPC 32) or TOC (for PowerPC 64) in assembly language source code, use the following macros:

Macro	Description
LOADPTR_SDA(*reg, symbol***)**	Loads the full 64 bit address of the symbol into the 64 bit register. Note: This macro is also available for use with PowerPC 32-bit systems. In those systems, it loads the 32-bit address into the register.
LOADVAR_SDA(*reg, symbol***)** and **STOREVAR_SDA(***v, sc, symbol***)**	Loads the 32 bit value at the address represented by the symbol into the register. This is mostly used to load variable values defined as the **int** or **unsigned int** type in a C source program from an assembly language program.
LOADVAR_LP_SDA(*reg, symbol***)** and **STOREVAR_LP_SDA(***v, sc, symbol***)**	Loads the 32-bit (for PowerPC 32) or 64-bit (for PowerPC 64) value at the address represented by the symbol into the register. This is mostly used to load variable values defined as the **long** or **unsigned long** type in a C source program from an assembly language program.

The macros ending with **_SDA** are semantically the same as the similarly named macros that do not end in **_SDA**. For example, both **LOADVAR** and **LOADVAR_SDA** are used to load the value of an **int** or an **unsigned int** value of a variable. The difference is how the address is obtained. The non-SDA macro gets the address of the symbol (variable) by using five instructions to calculate a 64-bit address for PowerPC 64, and two instructions to calculate the 32-bit address for PowerPC 32. This is absolute addressing. The **_SDA** variant obtains the address of the symbol (variable) using the SDA base for PowerPC 32 (which loads the value directly so that it does not use extra instructions for address loading) or the TOC base for PowerPC 64 (which uses two instructions for address loading). This is relative addressing.

For PowerPC 32, the SDA macros are accessing variable values residing directly in the SDA section. For PowerPC 64, they are accessing variable addresses residing in the TOC section and then using the addresses to load variable values. That is, for PowerPC 64, variables accessed by the SDA macros are still residing in the corresponding data sections; only their addresses are in the TOC section.

To load a per-CPU variable (that is, variables defined in the **WIND_VARS** structure and located in the **vxKernelVars[]** array) into a register in assembly language source code, use the following macros:

Macro	Description
_PPC_PER_CPU_ADRS_GET(r, *label***)**	Get the address of the *label* variable for the current CPU into the **r** register.
_PPC_PER_CPU_VALUE_GET(r, *label***)**	Get the value of the *label* variable (of the **int** type) for the current CPU into the **r** register.
_PPC_PER_CPU_VALUE_LP_GET(r, *label***)**	Load the value of the *label* variable (of the **long** type) for the current CPU into the **r** register.
_PPC_PER_CPU_VALUE_SET(v, sc, *label***)**	Set the value of the *label* variable (of the **int** type) for the current CPU with the value in the **v** register, overwriting the **sc** register.
_PPC_PER_CPU_VALUE_LP_SET(v, sc, *label***)**	Set the value of the *label* variable (of the **long** type) for the current CPU with the value in the **v** register, overwriting the **sc** register.
_PPC_PER_CPU_VALUE_AND_ADRS_GET(rv, ra, *label***)**	Get the value and the address of the *label* variable (of the **int** type) for the current CPU into the **rv** and **ra** registers, respectively.
_PPC_PER_CPU_VALUE_LP_AND_ADRS_GET(rv, ra, *label***)**	Get the value and the address of the *label* variable (of the **long** type) for the current CPU into the **rv** and **ra** registers, respectively.
_PPC_PER_CPU_VALUE_SET_ADRS_GET(v, ra, *label***)**	Set the value of the *label* variable (of **int** type) for the current CPU with the value in the **v** register. Also, puts the address of the *label* variable for the current CPU into the **ra** register.
_PPC_PER_CPU_VALUE_LP_SET_ADRS_GET(v, ra, *label***)**	Set the value of the *label* variable (of the **long** type) for the current CPU with the value in the **v** register. Also, puts the address of the *label* variable for the current CPU into the **ra** register.

6. POWERPC MEMORY MANAGEMENT

1. VxWorks Run-Time Support for MMU on PowerPC

Add MMU support for your PowerPC system to your VxWorks image project (VIP). VxWorks 7 supports the *dynamic model* for memory mapping.

The VxWorks kernel provides support for the MMU for PPCE500V2, PPCE500MC, PPCE6500, and PPC603 processors. To include this support, configure INCLUDE_MMU_BASIC in your VIP.

Dynamic Model for Memory Mapping

VxWorks 7 for PowerPC 32 supports the dynamic model for memory mapping. This model provides the ability to map physical memory in 4 KB pages using the remaining available TLB entries in a round-robin fashion.

Note: VxWorks 7 does not support the static model, because it needs to support the standard overlapped memory model.

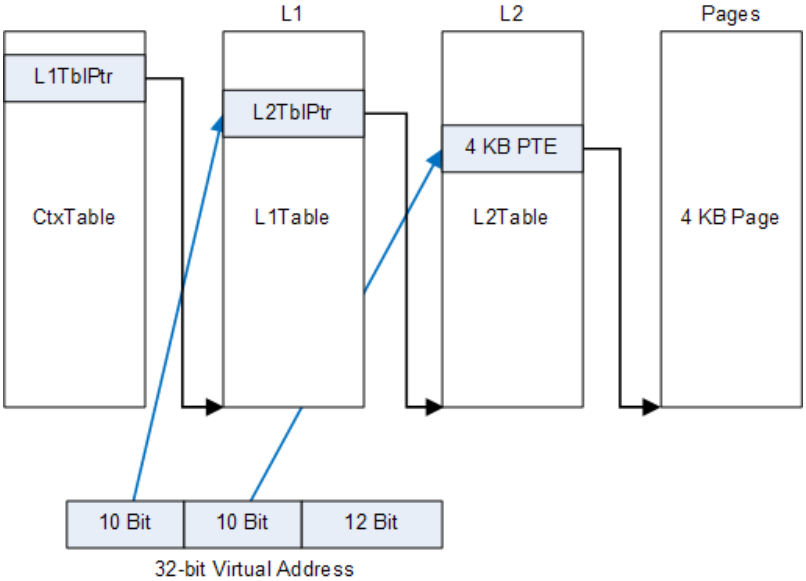
The dynamic mapping model allows memory to be mapped in 4 KB pages. The translation table is organized into two levels. The top level consists of an array of 1,024 Level 1 (L1) table descriptors; each of these descriptors can point to an array of 1,024 Level 2 (L2) table descriptors. All mapping attributes are defined in L2 descriptors (write-through/copy-back, cache-inhibited, guarded, execute, and write permissions).

The translation table size depends on the total memory to be mapped. The larger the memory to be mapped, the bigger the table.

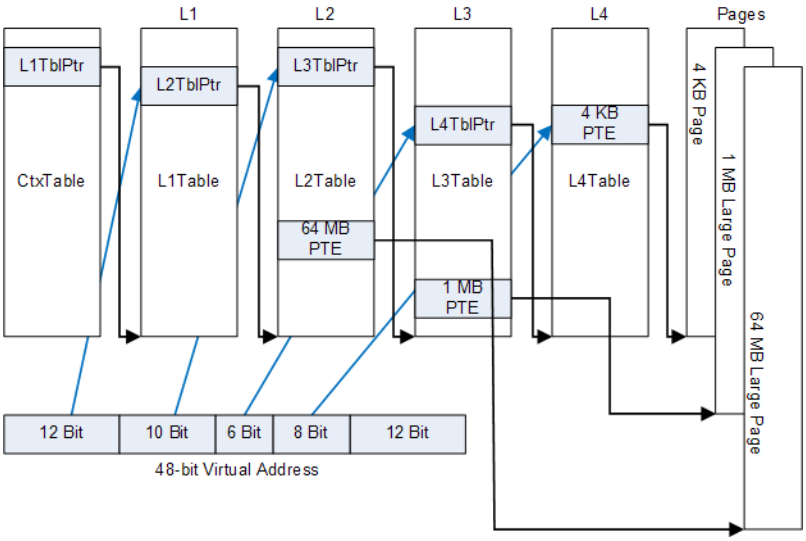
Note that VxWorks allocates page-aligned descriptor arrays from the heap at virtual memory initialization time. This results in a small amount of initial memory fragmentation.

The application programmer interface for this dynamic model is identical to the MMU translation model.

For PowerPC 32, each entry in the first level page table consists of a single pointer to a second level page table. Each entry in the second level page table contains an architecture-specific page table entry (PTE). Each PTE stores a physical address, protection information, and page size information. The following image illustrates this layout:



For PowerPC 64 systems, VxWorks must cover more virtual address bits on the system. VxWorks supports 48 virtual address bits which provides 256 TB of virtual address space. This is accomplished using a four level page table hierarchy as follows:



MMU Translation Model for PowerPC

VxWorks for PowerPC implementations share a common programming model for mapping 4 KB memory pages. PowerPC 64-bit also supports large page optimization with 1 MB and 64 MB pages.

All of the configuration constants defined in the *VxWorks 7 Platform Programmer's Guide: Kernel Virtual Memory Manager* are available for PowerPC virtual memory pages. The following information is specific to PowerPC systems:

- Use of the **MMU_ATTR_CACHE_DEFAULT** constant sets the cache to copy-back mode.
- **MMU_ATTR_CACHE_COHERENCY** and **MMU_ATTR_CACHE_GUARDED** are supported for PowerPC.

Cache coherency and guarded modes are controlled by the other constants. There is no default configuration, because each memory region may have specific requirements; see individual BSPs for examples.

> Note: For VxWorks SMP, **USER_D_CACHE_MODE** must not be defined as **CACHE_WRITETHROUGH**, nor may any of the system memory be configured with cache as **WRITETHROUGH**. However, **WRITETHROUGH** can be specified as needed for addresses corresponding to I/O device registers.

For more information on cache considerations, see PowerPC Cache Considerations on page 19.

Related information

Memory Mapping for PPCE500V2, PPCE500MC, PPCE6500, and PPC603 on page 17
Expose the Target Memory Map Programmatically on page 18

2. Memory Mapping for PPCE500V2, PPCE500MC, PPCE6500, and PPC603

Memory mapping for PowerPC-based systems varies, depending on the architecture.

For more information on physical addressing support, see PowerPC Architecture Information on page 3.

The PPCE500V2, PPCE500MC, and PPCE6500 are an implementation of the Book E processor specification. For 32-bit systems, the MMU is always active, and all addresses are translated either by a TLB0 (dynamic, fixed-4 KB-size TLB) or by a TLB1 (static, variable-size TLB) entry. This easily allows for a static map to be used for boot and basic operations when $MSR_{(IS,DS)}$ = (0,0) (VxWorks regards this as MMU disabled), and enables dynamic 4 KB page mapping when MSR_{IS} = 1 or MSR_{DS} = 1 (MMU enabled).

When an exception or interrupt is received, the $MSR_{(IS,DS)}$ bits are cleared. Code starts executing from the vector offset that is defined for the corresponding type of exception or interrupt. The $MSR_{(IS,DS)}$ bits are restored before control is transferred to the higher-level handler code. Prior to this restoration, any needed mappings for instruction and data accesses must be provided by static entries in TLB1.

The PPC603 is an implementation of the 32-bit PowerPC architecture. The MMU is always active (except when in an exception mode). All addresses are translated by a page table entry (PTE) (dynamic, fixed 4 KB size TLB). When the CPU enters into an exception mode, the $MSR_{(IR,DR)}$ bits are cleared automatically and the MMU is switched to real addressing mode translation.

For 64-bit PPCE500V2, PPCE500MC, and PPCE6500, some TLB1 entries are used for static mappings. The remaining TLB1 entries are used for page size optimizations.

Related information

VxWorks Run-Time Support for MMU on PowerPC on page 16
Expose the Target Memory Map Programmatically on page 18

3. Expose the Target Memory Map Programmatically

The BSP statically defines the memory regions that are available to VxWorks while it is running on this target. For example, on the **fsl_p1p2** BSP, the memory map for the **p2020rdb** board is defined in the file **p2020rdb.dts**.

```
memory {
    device_type = "memory";
    reg = <0x00000000 0x40000000>;
};
```

The memory map is also available programmatically to the VxWorks application, BSP and driver code. The VxWorks API function sysMemDescGet( ) must be called to retrieve the memory map configuration while the target is running.

For example:

```
LOCAL PHYS_MEM_DESC sysPhysRamDesc[32];
LOCAL STATUS success;

success = sysMemDesc(MEM_DESC_RAM, 0, sysPhysRamDesc);
```

Related information

VxWorks Run-Time Support for MMU on PowerPC on page 16
Memory Mapping for PPCE500V2, PPCE500MC, PPCE6500, and PPC603 on page 17

7. POWERPC CACHE CONSIDERATIONS

Most PowerPC processors contain an instruction cache and a data cache. In the default configuration, VxWorks enables both caches.

Cache support is configured in your VxWorks image project (VIP). Both data and instruction cache is enabled and cannot be disabled in either UP or SMP mode.

For most boards, the cache capabilities must be used with the MMU to resolve cache coherency problems. The page descriptor for each page selects the cache mode. This page descriptor is configured by filling the data structure **sysPhysMemDesc[ ]** defined in **sysLib.c**. (For more information about cache coherency, see the reference entry for **cacheLib**. For information about the MMU and VxWorks virtual memory, see the *VxWorks 7 Platform Programmer's Guide:* VxWorks 7 Memory Management. For MMU information specific to the PowerPC family, see Memory Mapping for PPCE500V2, PPCE500MC, PPCE6500, and PPC603 on page 17.

The state of both data and instruction caches is controlled by the WIMG information saved in the (translation lookaside buffer (TLB) where WIMG is defined as follows:

W

The **WRITETHROUGH** or **COPYBACK** attribute.

I

The cache-inhibited attribute.

M

The memory coherency required attribute.

G

The guarded memory attribute.

On PowerPC processors, cache flush at a specific address is usually performed by the **dcbst** instruction. Flushing of the entire cache usually involves loading from main memory over an address range. The starting address of the address range to load from is determined by the value stored in the variable **cachePpcReadOrigin**. The default value of **cachePpcReadOrigin** is 0x10000+cache line size; this value can be changed in the BSP.

During initialization of the MMU library, the MMU code provides the cache code with a suitably aligned address shortly after the MMU is initialized. **cachePpcReadOrigin** is set to a suitably aligned address within the first cacheable entry of **sysPhysMemDesc[ ]** that is of a sufficient size to accommodate the flush mechanism requirements. The required size is processor-dependent (typically, one and a half times the size of the cache for most processors). If the MMU is not configured, or if such a block of memory cannot be found, the default value of **cachePpcReadOrigin** is used. If your BSP overrides the default value of **cachePpcReadOrigin**, the BSP-supplied value is used in place of the default value.

cachePpcReadOrigin needs to point to cacheable memory in order for the load to properly displace modified entries in the cache that is flushed. If this scheme does not work for your target system for any reason, you must override **cachePpcReadOrigin** in the earlyInit( ) function in your BSP or the processor support library (PSL). (For more information on the PSL, see the VxWorks 7 BSP and Driver Guide.)

VxWorks SMP Considerations

If you are using VxWorks SMP on a PowerPC platform, the following restrictions apply:

- USER_D_CACHE_MODE must not be defined as **CACHE_WRITETHROUGH**, nor can any of the system memory be configured with cache as **WRITETHROUGH**.
- **WRITETHROUGH** can be specified as needed for addresses corresponding to I/O device registers.

WNDRVR

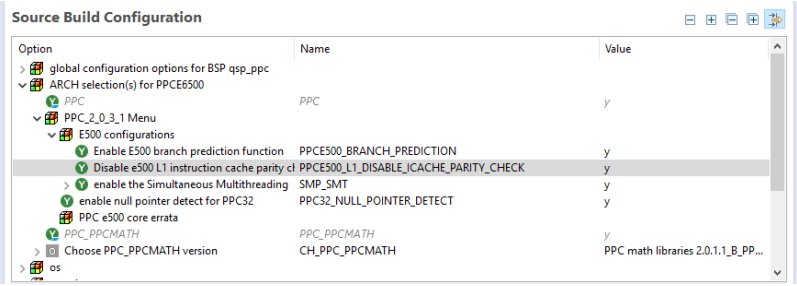
AIM Model for Caches

The architecture-independent model (AIM) for cache provides an abstraction layer to interface with the underlying architecture-dependent cache code. This allows uniform access to the hardware cache features that are typically CPU core specific. AIM for cache is for VxWorks internal use and does not change the VxWorks API for application development. For more information on the cache API, see the reference entry for **cacheLib**.

Disabling Parity Checking on the L1 Instruction Cache

You can configure a VSB to disable parity checking on the Level 1 (L1) instruction cache. Disabling parity checking may be the solution you need to improve peformance on older processors.

In the VSB **Source Build Configuration** window, enable the PPCE500_L1_DISABLE_ICACHE_PARITY_CHECK option as shown below.



8. VXWORKS COPROCESSOR SUPPORT FOR POWERPC

PowerPC coprocessors include floating-point, AltiVec, and signal processing engine (SPE). Enable default coprocessor support in your VxWorks source build (VSB) and image project (VIP).

Floating-Point VxWorks Source Build and Task Option Settings on page 21 The floating-point VxWorks source build (VSB) configuration determines the task option settings used by VxWorks when tasks are spawned.
Relationship between VSB and VIP Floating-Point Settings on page 22 The VxWorks image project (VIP) is affected by the floating-point configuration of the VxWorks source build (VSB).

1. Floating-Point VxWorks Source Build and Task Option Settings

The floating-point VxWorks source build (VSB) configuration determines the task option settings used by VxWorks when tasks are spawned.

Coprocessor abstraction decouples the core OS from the CPU-family-specific implementation of coprocessor features. Each architecture maps their coprocessors by logical number into the abstraction layer provided by the core OS.

Three coprocessor related options are available to the VSB. These are floating-point options which determine the default behaviour of the VxWorks kernel during context switches, and the GCC compiler options used when compiling code:

khard

All VxWorks tasks are spawned with VX_FP_TASK/VX_SPE_TASK set, and so the floating-point or SPE registers are always saved and restored for you on a task context switch. They are also saved and restored by VxWorks on an interrupt service routine (ISR) context switch. The GCC compiler is always run with the option -mhard-float which generates code to use the coprocessor. Macro **_WRS_CONFIG_FP_khard** is also set.

ksoft

User-mode RTP tasks are spawned with VX_FP_TASK/VX_SPE_TASK set, while all kernel tasks are spawned with this bit unset. Therefore, only user-mode tasks have floating-point or SPE registers saved and restored on context switches, this is never done for kernel tasks or ISRs. When building user-mode projects, GCC is run with the option -mhard-float so that RTPs use the coprocessor, while all kernel domain projects are compiled using the option -msoft-float and so do not. Macro **_WRS_CONFIG_FP_ksoft** is also set.

soft

All VxWorks tasks are spawned with VX_FP_TASK/VX_SPE_TASK unset, the floating-point or SPE registers are never saved and restored on a task or ISR context switch, and GCC is run with the option -msoft-float. The coprocessor is not used by default. Macro **_WTS_CONFIG_FP_soft** is also set.

For more details on these three options and what they mean, refer to VxWorks Source Build Floating Point Configuration on page 29 and the *VxWorks 7 C11, C++14, and Compiler Migration Reference*.

Table 1. PowerPC Coprocessor and Task Option Flag

Coprocessor Number	Name	Task Option Flag
1	Floating-Point	VX_FP_TASK
2	AltiVec	VX_ALTIVEC_TASK

Coprocessor Number	Name	Task Option Flag
3	SPE	VX_SPE_TASK

For more information on floating-point support, see VxWorks Support for the PowerPC Floating-Point Unit on page 24.

For more information on AltiVec support, see VxWorks Support for the PowerPC AltiVec on page 32.

For more information on SPE support, see VxWorks Support for the PowerPC SPE on page 40.

Parent topic: VxWorks Coprocessor Support for PowerPC on page 21

2. Relationship between VSB and VIP Floating-Point Settings

The VxWorks image project (VIP) is affected by the floating-point configuration of the VxWorks source build (VSB).

PPCE6500, PPCE5500, PPCE500MC and PPC60x Architectures

When the VSB is configured with processor support for PPCE6500, PPCE5500, PPCE500MC and PPC60x, then the coprocessor **Floating Point configuration** is always **_WRS_CONFIG_FP_khard** by default. Both the kernel and Real Time Process (RTP) applications will use hardware floating point, and no additional floating point components are added to the VIP that is based on this VSB.

If you change the VSB **Floating Point configuration** to **_WRS_CONFIG_FP_ksoft** then on creating the VIP, the component **INCLUDE_HW_FP** is added to the project for you. In this configuration, kernel functions use soft floating-point libraries, while RTP functions use hardware floating-point.

If you change the **Floating Point configuration** VSB option to **_WRS_CONFIG_FP_soft**, no component is added to the VIP and kernel and RTP tasks use software floating point libraries by default.

PPCE6500 Architecture and AltiVec

When your VSB is configured with processor support for PPCE6500, then the VIP can have the **INCLUDE_ALTIVEC** component added to it so that the kernel will support applications that are built to use AltiVec instructions. There is no dependency between the components **INCLUDE_ALTIVEC** and **INCLUDE_HW_FP** and AltiVec support functions independently from the floating-point and SPE VSB configuration. You must ensure that tasks built to use AltiVec instructions always have the **VX_ALTIVEC_TASK** task option set.

Specific compiler options must be used with projects that use AltiVec instructions. For more details, refer to PowerPC AltiVec Compiler Considerations on page 36.

PPCE500V2 Architecture

When your VSB is configured with processor support for PPCE500V2, then the default coprocessor setting is **_WRS_CONFIG_FP_khard**. In this configuration, both kernel and RTP functions use the PowerPC Signal Processing Engine (SPE) as a floating point coprocessor, and no floating point related components are added to the VIP.

If you change the VSB configuration to select **_WRS_CONFIG_FP_ksoft**, then when a VIP is created, the component **INCLUDE_SPE** is automatically added to it. In this configuration, kernel functions use soft floating-point libraries, while RTP functions use the hardware SPE.

If you change the **Floating Point configuration** VSB option to **_WRS_CONFIG_FP_soft**, no additional component is added to the VIP. All kernel and RTP tasks are set to use software floating point libraries by default.

Parent topic: VxWorks Coprocessor Support for PowerPC on page 21

9. POWERPC FLOATING-POINT CONSIDERATIONS

1. VxWorks Support for the PowerPC Floating-Point Unit

VxWorks includes support for the hardware floating-point unit available on certain PowerPC processors.

The floating-point unit (FPU) is available on PPCE500V2, PPCE500MC, PPCE6500, and PPC603.

For more information on PowerPC coprocessor support in VxWorks, see VxWorks Coprocessor Support for PowerPC on page 21 and VxWorks Source Build Floating Point Configuration on page 29

The FPU feature support for the VxWorks kernel depends on the VxWorks source build (VSB) **Floating Point configuration** option setting:

- Run-time detection of the FPU is possible using the fppProbe( ) function. This function is only available in ksoft mode, and used internally by VxWorks to prevent attempts to enable the FPU for a CPU that lacks such a unit.
- VxWorks tasks are automatically spawned with the **VX_FP_TASK/VX_SPE_TASK** option flag correctly setup based on the VxWorks source build (VSB) **Floating Point configuration** option. You will not need to change this again, unless you opt to override a soft floating-point system's behaviour. For further details, see Tune the Floating-Point Performance of the VxWorks System on page 30.
- Tasks cannot be spawned with float parameters. Only integer-sized parameters can be passed to taskSpawn( ).
- The VxWorks kernel automatically saves and restores all 32 FPU registers / SPE registers during every context switch when the VSB is set to khard, and also when the VSB is set to ksoft and user-mode context switches occur. The **FPSCR** is also saved in the context switch.
- The fppCtxShow( ) function displays values of the FPU registers in the shell. Only available in ksoft mode.

  Note: In khard mode, Wind River recommends you view the floating-point coprocessor hardware registers using the function ti( ).

- When the VSB is set to either ksoft or soft, and you opt to override the default software floating-point behaviour of the kernel, then you must use the fppSave( ) and fppRestore( ) functions to save and restore the FPU registers that are used from interrupt service routines (ISRs) (see example below). Before calling these functions, you must ensure that memory is allocated to store the values, and that the memory is aligned on an 8-byte boundary.

  Note: Do not confuse the hardware floating-point provided by the FPU with that provided by the SPE. You must use the speSave( ) and speRestore( ) functions instead to save and restore floating-point context. For more information, see VxWorks Support for the PowerPC SPE on page 40.

The following table summarizes the FPU-specific functions supported by VxWorks:

Table 1. FPU-Specific Functions

Function	Command Syntax	Description	Available
fppCtxShow( )	`[FP_CONTEXT *]`	Prints the contents of a FP context (usually called by other	ksoft

Function	Command Syntax	Description	Available
		functions, like the shell command ti).	
fppTaskRegsSet( )	`[task, FP_CONTEXT *]`	Sets the FPU registers of a task.	ksoft
fppTaskRegsGet( )	`[task, FP_CONTEXT *]`	Gets the FPU registers from a task TCB.	ksoft
fppProbe( )		Probes for the presence of an FPU.	ksoft
fppSave( )	`[FP_CONTEXT *]`	Saves FPU registers to memory.	ksoft, soft
fppRestore( )	`[FP_CONTEXT *]`	Restores FPU registers from memory.	ksoft, soft

Using Floating-Point in ISRs

To use floating-point in ISRs when the VSB is set to either ksoft or soft, you must ensure that the FPU is enabled in the MSR. The floating-point save function, fppSave( ), accomplishes this along with saving the previous floating-point context. The following example shows an ISR which uses the FPU properly:

```
void exampleFpIsr (void)
    {
    FP_CONTEXT fpContext;

    /*
     * FPU is not available at this point since MSR.FP is cleared
     * at interrupt entry.
     */

    /*
     * Save FPU registers.
     *
     * fppSave() sets MSR.FP within the function and leaves
     * it set.
     */

    fppSave (&fpContext);

    /*
     * Now you may call a routine to do any necessary processing using the
     * FPU.
     *
     * Note:
     * Do not use the FPU directly in this ISR, since the compiler
     * may generate instructions to save/restore FPU context in the
     * prologue/epilogue of this ISR.  Since this will be executed with
     * the MSR.FP bit cleared, an exception will be generated.
```

```
 */

myFpRoutine ();

/* restore FP registers */

fppRestore (&fpContext);
}
```

Note: If the VSB **Floating Point configuration** is set to **khard** then the save/restore operations are done automatically by the kernel for every ISR in the system. You can omit the code shown in the example above. However, if the VSB option is set to **ksoft** or **soft** and you have opted to use hardware floating point from your ISR anyway, then you must include the save/restore operations shown in the example above.

Related information

VxWorks Floating-Point Library Support for PowerPC on page 26
VxWorks Source Build Floating Point Configuration on page 29
Tune the Floating-Point Performance of the VxWorks System on page 30

2. VxWorks Floating-Point Library Support for PowerPC

VxWorks support for floating-point functions varies depending on your compiler choice and target architecture.

PPCE500V2, PPCE500MC, PPCE6500 and PPC603 Processors

The VxWorks floating-point library is used when the VxWorks source build (VSB) enables a floating point configuration of either **ksoft** or **soft**.

PPCE500V2, PPCE500MC, PPCE6500, and PPC603 processors support single-precision hardware floating-point instructions.

When using the GNU compiler, VxWorks provides a floating-point library that emulates the following mathematical functions. All ANSI floating-point functions have been optimized using libraries from U.S. Software.

The following double-precision functions are available:

acos( )	asin( )	atan( )	atan2( )
ceil( )	cos( )	cosh( )	exp( )
fabs( )	floor( )	fmod( )	log( )
log10( )	pow( )	sin( )	sinh( )
sqrt( )	tan( )	tanh( )	

The following single-precision functions are also available:

acosf( )	asinf( )	atanf( )	atan2f( )
ceilf( )	cosf( )	expf( )	fabsf( )
floorf( )	fmodf( )	logf( )	log10f( )
powf( )	sinf( )	sinhf( )	sqrtf( )
tanf( )	tanhf( )		

The following floating-point functions are not available on MPC85XX processors:

cbrt( )	infinity( )	irint( )	iround( )
log2( )	round( )	sincos( )	trunc( )
cbrtf( )	infinityf( )	irintf( )	iroundf( )
log2f( )	roundf( )	sincosf( )	truncf( )

e500v2 processors support double-precision hardware floating-point instructions. When using the GNU compiler, VxWorks includes double-precision hardware floating-point libraries in its math library. The following functions are available:

acos( )	asin( )	atan( )	atan2( )
ceil( )	cos( )	cosh( )	exp( )
fabs( )	floor( )	fmod( )	log( )
log10( )	pow( )	sin( )	sinh( )
sqrt( )	tan( )	tanh( )	

Note: IEEE754 support is not available.

Note: Single-precision hardware floating-point support is not available for e500v2 processors.

The P2020 includes additional hardware double-precision floating-point capabilities. The GNU compiler supports the hardware double-precision hardware capability for e500v2 processors. The required GNU compiler options are as follows:

-te500v2 -mfloat-gprs=double -mspe=yes -mabi=spe

When using double-precision hardware floating-point, this option replaces any specific SPE compiler flags.

Determine a Task's VX_FP_TASK Option State

Verify that VX_FP_TASK is enabled for your task from the VxWorks kernel shell. Assuming the component **INCLUDE_COPROCESSOR_SHOW** is included in the VIP, the ti ( ) function shows the state of a task's options.

For example, assuming that **taskA** has a task id of **0x40003df90**, verify that VX_FP_TASK is enabled as follows:

```
-> ti 0x40003df90

  NAME             TID       PRI   STATUS         PC          ERRNO  DELAY
----------  ---------------- --- ---------- ---------------- ------- -----
taskA              40003df90  50 PEND              18040b830       0     0

full task name : taskA
task entry     : taskA
task affinity  : 0x00000001
CPU Index      : -1 (Task Not Running)
process        : kernel
options        : 0x1009007
VX_SUPERVISOR_MODE  VX_UNBREAKABLE   VX_DEALLOC_STACK   VX_DEALLOC_TCB   VX_FP_TASK   VX_DEALLOC_E
XC_STACK
```

The task options indicate that VX_FP_TASK has been enabled for **taskA**.

Floating-Point Exceptions

Handling of floating-point exceptions is supported on PowerPC processors. By default, the floating-point exceptions are disabled on all available CPU cores.

To change this default setting for a task spawned with the VX_FP_TASK option, modify the values of the machine state register (MSR) and the floating-point status and control register (FPSCR) at the beginning of the task code.

- The MSR FE0 and FE1 bits select the floating-point exception mode.
- The FPSCR VE, OE, UE, ZE, XE, NI, and RN bits enable or disable the corresponding floating-point exceptions and rounding mode. (See **archPpc.h** for the macro **PPC_FPSCR_VE** and so forth.)

You access register values using the functions vxMsrGet( ) and vxFpscrGet( ), and you set register values using the functions vxMsrSet( ) and vxFpscrSet( ). You must only use the functions vxMsrSet( ) and vxFpscrSet( ) from a task spawned with the VX_FP_TASK option enabled.

CAUTION: **Tasks spawned without VX_FP_TASK must never change the CPU floating-point exception configuration.**

Any changes your task makes to the floating-point exception configuration will apply only to the particular CPU core that is currently executing your task. The floating-point exception status of the other CPU cores is not affected by your task. From the perspective of the VxWorks task, the floating-point exception will be enabled only in the task context of the one that enables floating-point exceptions.

For example, assume that **taskA** enables floating-point exceptions, and **taskB** leaves them disabled. If **taskA** is running on **core0**, then this floating-point configuration affects only **core0** and persists until the VxWorks scheduler preempts **taskA's** execution and schedules **taskB** to run on **core0** instead. At that point, because **taskB** was not spawned with VX_FP_TASK and so has not enabled floating-point exceptions in MSR and FPSCR, the context switch from **taskA** to **taskB** causes floating-point exceptions to be disabled again on **core0**. While **taskB** is the running task, floating-point exceptions remain disabled on **core0**. When the scheduler eventually reschedules **taskA** for execution, this time on **core4**, at that point **taskA's** floating-point registers are restored to their required state and floating-point exceptions are enabled in MSR and FPSCR on the CPU core. But this time, they are enabled on **core4** for the duration of **taskA's** execution.

Note: This behaviour is also equivalent on a uniprocessor VxWorks configuration where the kernel is only scheduling tasks to run on a single CPU core.

Related information

VxWorks Support for the PowerPC Floating-Point Unit on page 24
VxWorks Source Build Floating Point Configuration on page 29

Tune the Floating-Point Performance of the VxWorks System on page 30

3. VxWorks Source Build Floating Point Configuration

Options in the VxWorks source build (VSB) give you fine-grained control over the impact that floating point processing has on system throughput and latency.

The behaviour of the VxWorks system, as it relates to floating-point operations, has two important aspects:

1. whether the compiler generates code that calls the hardware floating point coprocessor, or the software floating point library, to deal with variables of type float and double.
2. whether or not VxWorks saves the state of the CPU floating point registers (or equivalent SPE registers) for a particular VxWorks task or interrupt service routine (ISR) context.

These aspects apply individually to tasks in the kernel and the RTP domains.

On PowerPC architecture, VxWorks manages the floating point behaviour of these elements and these domains from the VSB option, **Select the Floating Point configuration**. This option has three possible settings:

- **khard**
- **ksoft**
- **soft**

This VSB option controls when the compiler is run with the -mhard-float option to generate floating point coprocessor code, and when the kernel automatically calls fppSave( ) (or the SPE equivalent functions) on switching VxWorks system contexts.

khard on page 29
ksoft on page 30
soft on page 30

Related information
VxWorks Support for the PowerPC Floating-Point Unit on page 24
VxWorks Floating-Point Library Support for PowerPC on page 26
Tune the Floating-Point Performance of the VxWorks System on page 30

3.1. khard

This option gives maximum hardware assisted floating point performance and system throughput, at the cost of increased kernel latency.

The **VX_FP_TASK** option bit is set automatically for every kernel and real time process (RTP) task created by the system. The compiler is run with the option -mhard-float to generate hardware floating point compatible code for kernel and RTP domains, and the VxWorks kernel saves and restores the state of the CPU floating point registers on every task and interrupt service routine (ISR) context switch.

Note: On the PPCE500V2 architecture, the VX_FP_TASK option bit is not involved. The VX_SPE_TASK option bit is used instead. Also, the SPE registers are used, not the floating point registers.

Parent topic: VxWorks Source Build Floating Point Configuration on page 29

3.2. ksoft

This option balances floating point performance and system throughput with minimal kernel latency.

The **VX_FP_TASK** option bit is not set for any kernel task, it is just set for RTP tasks. The compiled code only uses hardware floating point from RTP tasks, the kernel domain uses the VxWorks software floating point library API. Also, the VxWorks kernel only saves and restores CPU floating point registers for RTP tasks, not kernel tasks or ISRs.

Note: On the PPCE500V2 architecture, the VX_FP_TASK option bit is not involved. The VX_SPE_TASK option bit is used instead. Also, the SPE registers are used, not the floating point registers.

Parent topic: VxWorks Source Build Floating Point Configuration on page 29

3.3. soft

This option gives the slowest floating point performance and lowest system throughput, combined with minimal VxWorks kernel latency. Depending on the constraints of your application, this option may give the best real-time performance.

The **VX_FP_TASK** option bit is not set for any kernel or RTP task. Further, the compiler is always given the -msoft-float option, and so it generates code that only invokes VxWorks software floating-point library API functions when float and double variables are used. Further, the CPU floating point registers are not saved and restored by default in any system context.

Note: On the PPCE500V2 architecture, the **VX_SPE_TASK** option bit replaces **VX_FP_TASK**, and the SPE registers are the ones that are not saved and restored.

Parent topic: VxWorks Source Build Floating Point Configuration on page 29

4. Tune the Floating-Point Performance of the VxWorks System

VxWorks gives you the ability to finely tune the throughput and real-time behaviour of the system as it relates to floating-point processing.

Determine the Setting of the Task Option Flags

Verify that VX_FP_TASK is enabled for your task from the VxWorks kernel shell. Assuming the component **INCLUDE_COPROCESSOR_SHOW** is included in the VIP, the ti ( ) function shows the state of a task's options.

For example, assuming that **taskA** has a task id of **0x40003df90**, verify that VX_FP_TASK is enabled as follows:

```
-> ti 0x40003df90

  NAME             TID       PRI   STATUS         PC          ERRNO  DELAY
----------  ---------------- --- ---------- ---------------- ------- -----
taskA              40003df90  50 PEND              18040b830       0     0

full task name : taskA
task entry     : taskA
task affinity  : 0x00000001
CPU Index      : -1 (Task Not Running)
process        : kernel
options        : 0x1009007
```

```
VX_SUPERVISOR_MODE  VX_UNBREAKABLE   VX_DEALLOC_STACK   VX_DEALLOC_TCB   VX_FP_TASK   VX_DEALLOC_E
XC_STACK
```

The task options indicate that VX_FP_TASK has been enabled for **taskA**.

Use the Task Option Flags to Tune Performance

The VxWorks source build (VSB) **Floating Point configuration** setting determines the default behaviour of the VxWorks system. While the khard setting increases system throughput by using the coprocessor on floating-point math, it increases system latency by saving and restoring the coprocessor registers on every system context switch. The ksoft setting reduces this added latency by limiting it to user-mode tasks only, and the soft option abandons use of the coprocessor altogether and so never saves and restores its register set. There is an element of fine tuning available to you in the selection of this option, yet you are only defining the default behaviour of the system with it. An even more finely-grained tuning approach is also available to you over and above this default setting.

Depending on the design of your application, you may have a small subset of math heavy tasks that require floating-point coprocessor offload to be enabled for them. However, the rest of your system may not actively use or benefit from floating-point coprocessor offloading in any way. In this scenario, it makes sense to set the default floating-point configuration to soft in the VxWorks source build (VSB) as this reflects the system's general behaviour. However, for the particular math heavy application tasks, whether coded in C/C++ or assembly language, you can also:

1. adjust the setting of the application task's **VX_FP_TASK/VX_SPE_TASK** bit. VxWorks initialised this to be unset by default. Change this default setting by calling taskOptionsSet( ) so that the task's floating-point registers will be properly saved and restored during context switches between the application tasks.
2. adjust the tool flags for the math heavy application project so that the compiler is run with the option -mhard-float. You must ensure that only the code running in the floating-point coprocessor enabled tasks are compiled with this option. All other tasks must have the compiler setting of -msoft-float applied to their compile lines.

   By targeting the floating-point configuration of the system in this way, you are minimizing system latency while maximising throughput where required.

   Note: You must never use taskOptionsSet( ) to unset **VX_FP_TASK/VX_SPE_TASK** if VxWorks has set it by default. If hardware floating-point is enabled in the VSB, using khard or ksoft, then you cannot disable it on a task by task basis. Instead, set the VSB to soft, and enable hardware floating-point support on a task by task basis.

Note: In some scenarios, the VxWorks GNU compiler will generate float instructions during compilation. This can cause issues if your VSB is set to a **soft** configuration. If your application task's math operations do not appear to generate consistently correct results, it is likely that the processor floating point registers are being used by your application task and then unexpectedly overwritten again by other tasks when the kernel reschedules them. To resolve this situation, adjust the taskSpawn( ) call for the VxWorks task that is generating incorrect math results and add **VX_FP_TASK** as a task option. This will ensure that, if the floating point registers are being used by your application task, they will be protected when the kernel reschedules tasks. Your application task should then generate consistently correct math results.

Related information

VxWorks Support for the PowerPC Floating-Point Unit on page 24
VxWorks Floating-Point Library Support for PowerPC on page 26
VxWorks Source Build Floating Point Configuration on page 29

10. POWERPC ALTIVEC SUPPORT CONSIDERATIONS

1. VxWorks Support for the PowerPC AltiVec

VxWorks includes support for the PowerPC AltiVec feature for PPCE6500 processors.

AltiVec is a vector coprocessor and PowerPC instruction set extension introduced on the MPC74XX family of processors. (The PowerPC E6500 processor includes similar functionality and are treated as AltiVec-enabled processors by VxWorks.) VxWorks treats AltiVec as an extension to the PPCE6500 core; that is, a PPCE6500 binary image can, in certain situations, run without modification on any AltiVec part, but the image does not provide access to, or control of, the AltiVec unit itself.

For more information on PowerPC coprocessor support in VxWorks, see VxWorks Coprocessor Support for PowerPC on page 21

The VxWorks implementation of AltiVec support includes:

- VxWorks run-time support for AltiVec.
- Enabling AltiVec support.
- Compiling modules that use the AltiVec unit.
- Debugging extensions for AltiVec.
- Workbench 4 tool support.
- Known problems with C++ mixed linking of AltiVec and non-AltiVec modules.

The following features are supported for the AltiVec unit by the VxWorks kernel:

- Run-time detection of the AltiVec unit is possible using the altivecProbe( ) function. This function is used internally by VxWorks to prevent attempts to enable AltiVec for a CPU that lacks such a unit. This allows a single build of a VxWorks kernel to run on boards that support both AltiVec and non-AltiVec parts.
- Tasks that use the AltiVec unit must be spawned with the VX_ALTIVEC_TASK option flag set.
- Tasks created without the VX_ALTIVEC_TASK option that use AltiVec instructions incur an AltiVec Unavailable Exception error, and the task is suspended.
- Tasks cannot be spawned with vector parameters. Only integer-sized parameters can be passed to taskSpawn( ).
- The altivecTaskRegsShow( ) function displays values of AltiVec registers in the shell.
- The altivecSave( ) and altivecRestore( ) functions save and restore AltiVec register contents from memory. These functions can be called from interrupt handlers (see example below). Before calling these functions, you must ensure that memory has been allocated to store the values, and that the memory is aligned on a 16-byte boundary.

AltiVec-Specific Functions

The AltiVec-specific functions shown in the following table are available in VxWorks:

Table 1. AltiVec-Specific Functions

Function	Command Syntax	Description
altivecInit( )		Initializes AltiVec coprocessor support.
altivecTaskRegsShow( )	[*task*]	Prints the contents of the AltiVec registers of a task.

Function	Command Syntax	Description
altivecTaskRegsSet( )	[*task* , ALTIVECREG_SET *]	Sets the AltiVec registers of a task.
altivecTaskRegsGet( )	[*task* , ALTIVECREG_GET *]	Gets the AltiVec registers from a task TCB.
altivecProbe( )		Probes for the presence of an AltiVec unit.
altivecSave( )	[ALTIVEC_CONTEXT *]	Saves vector registers to memory.
altivecRestore( )	[ALTIVEC_CONTEXT *]	Restores vector registers from memory.
altivecSetNonJava( )		Sets non-Java mode for AltiVec tasks.
vec_malloc( )	size_t	Returns a 16-byte aligned pointer for an object of a given size.
vec_calloc( )	size_t *nObj* , size_t *size*	Returns a 16-byte aligned pointer for an array of *nObj* objects each of size *size* , initialized to 0.
vec_realloc( )	void *p, size_t *nbytes*	Increases the size of a 16-byte aligned buffer to *nbytes* .
vec_free( )	void *p	Deallocates the memory area pointed to by **p**.

Note: Memory allocation in VxWorks for PPCE6500 is always 16-byte aligned; vec_malloc( ), vec_calloc( ), and vec_realloc( ) are aliases for alloc( ).

Using AltiVec in ISRs

To use AltiVec in ISRs, it is necessary to enable AltiVec through the MSR in addition to saving and restoring the AltiVec registers. The following example shows an ISR which uses AltiVec properly:

```
void exampleAltivecIsr (void)
    {
    UINT32      msr;
    ALTIVECREG_SET altivecRegSet;

    /* save AltiVec registers */

    altivecSave (&altivecRegSet);

    /*
     * Set MSR.VEC here since this bit is always cleared at interrupt
```

```
     * entry.
     *
     * altivecSave() sets MSR.VEC within the function but it is restored
     * before returning, thus MSR.VEC will be cleared at this point.
     */

    vxMsrSet ((msr = vxMsrGet ()) | _PPC_MSR_VEC);

    /*
     * Now you may call a routine to do any necessary processing using the
     * AltiVec.
     *
     * Note:
     * Do not use the AltiVec directly in this ISR, since the compiler
     * may generate instructions to save/restore AltiVec context in the
     * prologue/epilogue of this ISR.  Since this will be executed with
     * the MSR.VEC bit cleared, an exception will be generated.
     */

    myAltiVecRoutine ();

    /*
     * The MSR will be restored when it returns from interrupt.
     * However, we restore the MSR and disable AltiVec prior
     * to return from this ISR to be safe.
     */

    vxMsrSet (msr);

    /* restore AltiVec registers */

    altivecRestore (&altivecRegSet);
    }
```

Java Mode

The *Power ISA(TM) Version 2.04: Floating-Point Overview* specifies that, by default, the AltiVec unit "conforms to the Java Language Specification" with respect to the floating-point model. This means that any denormalized input or output values to certain vector floating-point instructions generate an AltiVec Assist Interrupt. By default, VxWorks configures this behavior, but it does not install a handler for this interrupt. Instead, this implementation is left to the user.

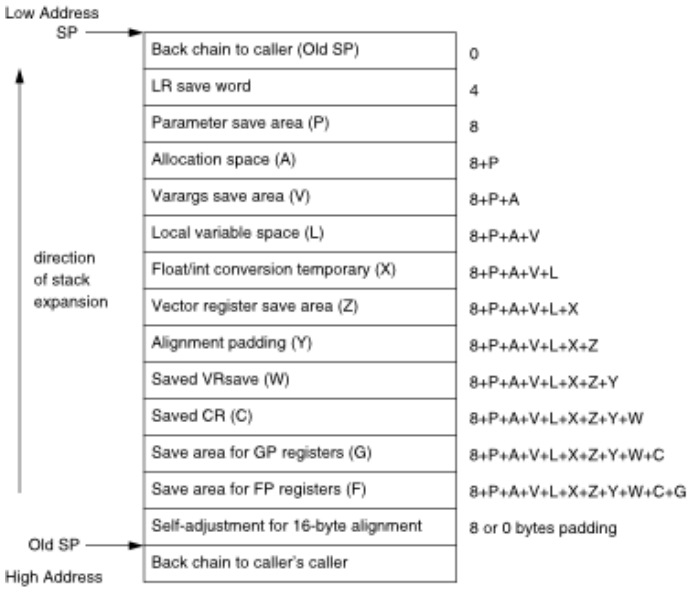
If you do not wish to install a handler for the AltiVec Assist Interrupt, you can instead configure the unit for non-Java mode. In this mode, denormalized values do not generate an interrupt, but instead are replaced with the appropriately signed zero. A global function, altivecSetNonJava( ) is available to set this mode. You can call this function yourself, prior to spawning AltiVec-enabled tasks, or you can configure it into the system as part of your VxWorks image project (VIP) using the configuration parameter ALTIVEC_NON_JAVA_MODE, which is part of the INCLUDE_ALTIVEC component.

Stack Frame Considerations for PowerPC 32

The stack frame for functions using the AltiVec registers adds the following areas to the standard EABI frame:

- A vector register save area (32 * 16 bytes).
- Alignment padding (always zero bytes because the frame is always 16-byte aligned).
- A saved VRSAVE register (4 bytes).

The stack frame layout for functions using the AltiVec registers is shown in the following figure (non-AltiVec stack frames are not affected).



Stack Frame Considerations for PowerPC 64

For information on the stack frame for PowerPC 64, see the *Power Architecture 64-Bit ELF V2 ABI Specification* available from:

http://openpowerfoundation.org/technical/technical-resources/technical-specifications/

Vector Data Type Considerations

The AltiVec specification adds a new family of vector data types to the C language. vector types are 128 bits long, and are used to manipulate values in AltiVec registers. Under control of a compiler option, vector is now a keyword in the C and C++ languages. The AltiVec programming model introduces five new keywords as simple type-specifiers:

- vector
- __vector
- pixel
- __pixel
- bool

> Note: In previous versions of the GCC compiler, **vector** was used as both a C++ class name and a C variable name in the VxWorks header files and some BSP source files, and so conflicted with the vector keyword. This conflict does not occur when using the current GCC 8.1 compiler. However, where possible, use __vector rather than vector in VxWorks code anyway as a precaution.

The *AltiVec Technology Programming Interface Manual* also specifies vector conversions for formatted I/O. VxWorks SR0600 does not support the new formatted input and output of vector data types.

Related information

PowerPC AltiVec Compiler Considerations on page 36

C++ Exception Handling with AltiVec on page 37

2. PowerPC AltiVec Compiler Considerations

The GNU compiler provides support for AltiVec. You should be aware of AltiVec-specific compiler features and limitations during your development.

Modules that use the AltiVec registers and instructions must be coded and compiled as follows:

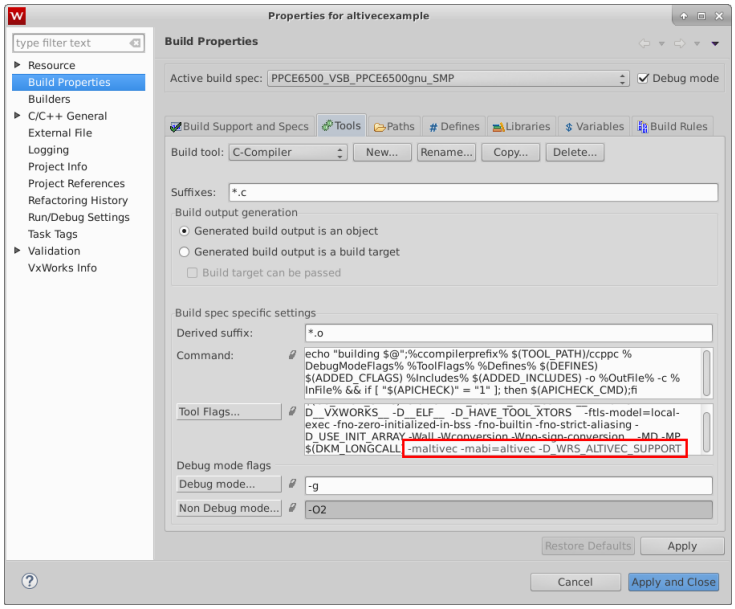
1. Source files must always use the following header file so that AltiVec vector types and intrinsics (functions that map directly to low-level AltiVec instructions) are included:

```
#include <altivec.h>
```

2. From the compiler command line, the following GCC compiler options must be specified:

```
-maltivec -mabi=altivec -D_WRS_ALTIVEC_SUPPORT
```

Note: To add these compiler options to a project from Workbench, locate the project **Tool Flags...** field and add them there.



Note: To add these compiler options automatically from the project makefile, you can use the Wind River supplied macro called **OPTION_ALTIVEC_C**. For example:

```
ADDED_CFLAGS+=$(OPTION_ALTIVEC_C)
```

EABI Compliance

The version of the GNU compiler included with this VxWorks release is fully compliant with the POWER AltiVec ISA EABI specification.

CAUTION: **Examples of commonly used AltiVec-enabled functions are the printf( ) and scanf( ) family of functions. Applications calling these functions with more than eight integer-class or more than eight floating-point arguments may behave unpredictably.**

Note: The GNU compiler may generate AltiVec instructions when -maltivec -mabi=altivec -D_WRS_ALTIVEC_SUPPORT is specified in the source code, even when the source code does not specify AltiVec calls, use reserved keywords, or include **altivec.h** files. This may cause any task that uses objects compiled with -maltivec -mabi=altivec to generate a "coprocessor not supported" exception if the task is not spawned with VX_ALTIVEC_TASK.

Related information

VxWorks Support for the PowerPC AltiVec on page 32
C++ Exception Handling with AltiVec on page 37

3. C++ Exception Handling with AltiVec

Throwing C++ exceptions between modules compiled with different compiler flags can result in unexpected behavior. You should be aware of this behavior during your development.

C++ exceptions save register state. Modules compiled with AltiVec support (using -maltivec -mabi=altivec) save all non-volatile AltiVec registers, but modules compiled without AltiVec support do not save any AltiVec registers. If a C++ exception is thrown from an AltiVec-enabled module, caught by a non-AltiVec enabled handler, and then thrown from there to an AltiVec-enabled handler that alters the AltiVec registers, it is possible to corrupt the saved AltiVec state. In particular, the non-volatile vector registers (v20 through v31) can be corrupted.

C++ Exception Handling Example

This example consists of a program composed of two files, **myAltivec1.cpp** and **myAltivec2.cpp**. Because **myAltivec2** is compiled with the -maltivec -mabi=altivec option, it is considered AltiVec code. **myAltivec1** is compiled without the -maltivec -mabi=altivec option, so it is considered non-AltiVec code.

The example takes program flow across the two modules. It is also contrived to make intelligent guesses about the compiler register allocation strategy. The output is incorrect when one of the files is compiled without the -maltivec -mabi=altivec option.

The **myAltivec1.cpp** file is as follows:

```
#include <vxWorks.h>
#include <stdio.h>
#include <stdlib.h>
#include <altivec.h>

/* using namespace std */

extern "C" int printf (const char *fmp, ...);
extern void bar ();

void foo ()
```

```
    {
    try
        {
        bar ();
        }
    catch (...)
        {
        }
    }
```

The **myAltiVec2.cpp** file is as follows:

```
#include <vxWorks.h>
#include <stdio.h>
#include <stdlib.h>
#include <altivec.h>

extern "C" int printf (const char *fmp, ...);
extern void foo ();

void bar ()
    {
    /* use a non-volatile vector register */
    asm ( "vspltisw 24,0" );        /* v24 <- (0,0,0,0) */
    }

void Start ()
    {
    /* use a non-volatile vector register v24 */
    __vector signed long local = {-1, -1, -1, -1};

    asm ( "vspltisw 24,15" );       /* v24 <- (15, 15, 15, 15) */
    foo ();

    /* continue using the non-volatile vector registers */
    asm ( "addi 9, 31, 32" );       /* local <- v24 */
    asm ( "stvx 24, 0, 9" );

    printf ("Finally, local = (&#037;vld)\n", local);
    }
```

1. Reproduce the unexpected behavior.
   To produce a partially linked object **myAltivec2.o**, compile the two files with the following commands:

```
% ccppc -mcpu=e6500 -c myAltivec1.cpp
% ccppc -mcpu=e6500 -nostdlib -maltivec -mabi=altivec -r myAltivec1.o myAltivec2.cpp
```

2. Download **myAltivec2.o** to a target, and execute the Start function.

```
-> Start
Finally, local = (0,0,0,0)
->
```

The foo( ) function in **myAltivec1.cpp** is non-AltiVec code. Therefore, the **try...catch** block in foo( ) does not save and restore the AltiVec context. Within the **try...catch** block, the call to bar( ) alters the value of vector register v24. Because **myAltivec1.cpp** does not save AltiVec context, the value 0 in v24 assigned by bar( ) remains unchanged when the program flow returns to Start. The original value 15, assigned before the call to bar( ), is now corrupted. Hence, the incorrect output, **local = (0,0,0,0)**.

3. Correct the behavior by compiling both modules with the -maltivec -mabi=altivec option.

```
% ccppc -mcpu=e6500 -nostdlib -maltivec -mabi=altivec -r myAltivec1.cpp myAltivec2.cpp -o myAlt
ivec2.o
```

Download **myAltivec2.o** to a target and execute the Start function.

```
-> Start
Finally, local = (15,15,15,15)
->
```

Because both modules now have AltiVec code (compiled with the -maltivec -mabi=altivec option), the **try...catch** block in foo( ) now saves and restores the AltiVec context. The value 15 originally assigned in Start is faithfully restored by foo( ) when it returns.

Related information

VxWorks Support for the PowerPC AltiVec on page 32
PowerPC AltiVec Compiler Considerations on page 36

11. POWERPC SIGNAL PROCESSING ENGINE (SPE) CONSIDERATIONS

1. VxWorks Support for the PowerPC SPE

VxWorks includes support for the PowerPC signal processing engine (SPE). The SPE is a SIMD processing unit with a PowerPC instruction set extension introduced on the MPC85XX family of processors.

> Note: Although PPCE500MC and PPCE6500 are variants of the MPC85XX, they do not include the SPE. Instead, they support PowerPC 60X-style hardware floating-point.

For more information on PowerPC coprocessor support in VxWorks, see VxWorks Coprocessor Support for PowerPC on page 21

The VxWorks implementation of SPE support includes:

- VxWorks run-time support for SPE.
- SPE EABI stack frame support.
- Compiling modules that use the SPE unit.
- Workbench 4 tool support.

The following features are supported for the SPE unit by the VxWorks kernel:

- The SPE unit initialization speInit( ) is performed by the usrSpeInit( ) function in:

  installDir **/vxworks-7/pkgs_v2/os/arch/ppc-***x.x.x.x* **/kernel/e500/cfg/usrSpe.c**

  Typically, this is called by the usrRoot( ) function if INCLUDE_SPE is defined.
- Run-time detection of the SPE unit is possible using the speProbe( ) function. This function is used internally by VxWorks to prevent attempts to enable SPE for a CPU that lacks such a unit.
- Tasks that use the SPE unit must be spawned with the VX_SPE_TASK option flag set.
- Tasks created without the VX_SPE_TASK option that use SPE instructions incur an SPE Unavailable Exception error, and the task is suspended.
- Tasks cannot be spawned with vector parameters. Only integer-sized parameters can be passed to taskSpawn( ).
- The MPC85XX processor's upper 32 bits in the general purpose registers are saved and restored as part of the task context. The VxWorks kernel saves and restores all 32 SPE register extensions when switching between SPE contexts. The SPEFSCR and the accumulator are also saved in the context switch.
- The speTaskRegsShow( ) function displays the values of all 64 bits of the general purpose registers in the shell.
- The speSave( ) and speRestore( ) functions save and restore the upper 32 bits of the general purpose register contents from memory. These functions can be called from interrupt handlers (see Using SPE in ISRs on page 41). Before calling these functions, you must ensure that memory is allocated to store the values, and that the memory is aligned on a 32-bit boundary.

SPE-Specific Functions

The following table summarizes the SPE-specific functions supported by VxWorks:

Table 1. SPE-Specific Functions

Function	Command Syntax	Description
speInit( )		Initializes SPE APU support.
speTaskRegsShow( )	[*task*]	Prints the contents of the SPE registers of a task.
speTaskRegsSet( )	[*task* , `SPEREG_SET *`]	Sets the SPE registers of a task.
speTaskRegsGet( )	[*task* , `SPEREG_SET *`]	Gets the SPE registers from a task TCB.
speProbe( )		Probes for the presence of an SPE unit.
speSave( )	`[SPE_CONTEXT *]`	Saves upper GPR registers to memory.
speRestore( )	`[SPE_CONTEXT *]`	Restores registers from memory.

Using SPE in ISRs

To use SPE in ISRs, it is necessary to enable the SPE through the MSR in addition to saving and restoring the SPE registers. The following example shows an ISR which uses the SPE properly:

```
void exampleSpeIsr (void)
    {
    UINT32     msr;
    SPEREG_SET speRegSet;

    /* save SPE registers */

    speSave (&speRegSet);

    /*
     * Set MSR.SPE here since this bit is always cleared at interrupt
     * entry.
     *
     * speSave() sets MSR.SPE within the function but it is restored
     * before returning, thus MSR.SPE will be cleared at this point.
     */

    vxMsrSet ((msr = vxMsrGet ()) | _PPC_MSR_SPE);

    /*
     * Now you may call a routine to do any necessary processing using the
     * SPE.
     *
     * Note:
```

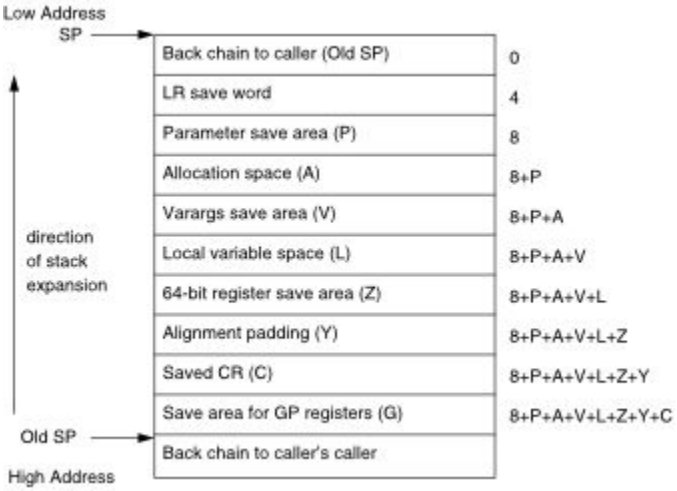
```
     * Do not use the SPE directly in this ISR, since the compiler
     * may generate instructions to save/restore SPE context in the
     * prologue/epilogue of this ISR.  Since this will be executed with
     * the MSR.SPE bit cleared, an exception will be generated.
     */

    mySpeRoutine ();

    /*
     * The MSR will be restored when it returns from interrupt.
     * However, we restore the MSR and disable SPE prior
     * to return from this ISR to be safe.
     */

    vxMsrSet (msr);

    /* restore SPE registers */

    speRestore (&speRegSet);
    }
```

Stack Frame Considerations

The stack frame for functions using the whole of the 64-bit general purpose registers adds the following areas to the standard EABI frame:

- A 64-bit register save area (32 * 8 bytes).
- Alignment padding (which is always zero bytes because the frame is always 8-byte aligned).

The stack frame layout for functions using the upper 32 bits of the general purpose registers is shown in the following figure (non-SPE stack frames are not affected).



The required alignment for the SPE EABI specification is 16 bytes. Therefore, you can call functions compiled for SPE from certain other PowerPC EABI-compliant code that assumes an 8-byte alignment for the stack boundary. However, the converse does not hold true and you may get undefined results.

Vector Data Type Considerations

The SPE specification adds a new family of vector data types to the C language. These data types are 64-bit entities which have other data types embedded in them. The new entities are:

- **__ev64_u16__**
- **__ev64_s16__**
- **__ev64_u32__**
- **__ev64_s32__**
- **__ev64_u64__**
- **__ev64_s64__**
- **__ev64_fs__**

The type **__ev604_opaque__** represents any of the above types.

The *SPE Programming Interface Manual* also specifies vector conversions for formatted I/O. VxWorks SR0600 does not support the formatted input and output of vector data types.

Compiler Considerations

The GNU compiler included with this release of VxWorks is fully compliant with the NXP SPE EABI specification.

Related information

PowerPC SPE Architecture Considerations for VxWorks on page 43

2. PowerPC SPE Architecture Considerations for VxWorks

When working with VxWorks and the PowerPC SPE, there are certain architecture characteristics that you should keep in mind.

SPE Exceptions Under Likely Overflow/Underflow Conditions

The signal processing engine (SPE) unit on the MPC85XX processors provides floating-point support for scalar or vector quantities. Some of these instructions generate an exception (if SPEFSCR is set accordingly) and return a pre-determined value if an overflow or underflow is likely, even though the actual result does not cause an overflow or underflow. The action needed to handle such a condition is application dependent. Thus, the user must set SPEFSCR accordingly and handle the erroneous result. The instructions that exhibit this behavior include: **efsadd**, **efssub**, **efsmul**, **efsdiv**, **evfsadd**, **evfssub**, **evfsmul**, and **evfsdiv**.

Note: This feature is not supported for PPCE500MC, PPCE6500, or PPC603 processors.

SPE Unavailable Exception in Relation to Task Options

The SPE on the PPCE500V2 and MPC85xx processors does not implement the standard PowerPC floating-point feature. The SPE implements its own floating-point instruction set. While the hardware supports only single-precision floating-point computation, there are two kinds of floating-point instructions:

- Scalar floating-point, which uses only the lower 32 bits of a general purpose register (GPR).
- Vector floating-point, which uses all 64 bits of a GPR.

VX_FP_TASK corresponds to the scalar floating-point, while **VX_SPE_TASK** corresponds to the vector floating-point. The difference between spawning a task with **VX_FP_TASK** and **VX_SPE_TASK** is that the task that is spawned with **VX_SPE_TASK** saves and restores the upper 32 bits in the GPRs during context switch, by means of task hooks.

Because both kinds of floating-point instructions require the use of the SPE coprocessor, the MSRSPE bit is enabled when either options is specified for the task. The following are some of the behaviors that result from this semantic:

- Tasks spawned with **VX_FP_TASK** but without **VX_SPE_TASK** do not save and restore the upper 32 bits of GPRs upon context switch.
- Tasks spawned with either **VX_FP_TASK** or **VX_SPE_TASK** are not able to generate the SPE unavailable exception when executing any SPE vector instructions.
- Tasks that use both scalar and vector floating-point instructions can only be spawned with **VX_SPE_TASK**. However, as a good programming practice, you should regard scalar floating-point as associated with **VX_FP_TASK**.

Programmatically, VxWorks makes no distinction between a task spawned with **VX_SPE_TASK**, and a task spawned with both **VX_SPE_TASK** and **VX_FP_TASK**. However, any debugging information shows the corresponding options as specified during task creation.

Note: When compiling with CPU set to PPCE500V2 and TOOL set to gnu, **VX_FP_TASK** behaves like **VX_SPE_TASK**. When the task is spawned, **VX_SPE_TASK** is set if **VX_FP_TASK** is already set.

Note: This feature is not supported for PPCE500MC, PPCE6500, or PPC603 processors.

Related information

VxWorks Support for the PowerPC SPE on page 40

WNDRVR

12. POWERPC BRANCHING CONSIDERATIONS

1. Branch Prediction on PPCE500V2, PPCE500MC, and PPCE6500

Branch prediction is available for the PPCE500V2, PPCE500MC, and PPCE6500 CPUs. However, you must ensure that the memory configuration is correct.

This includes proper configuration of the DDR controller, local access windows (LAWs), and TLBs. Incorrect configuration can result in reliability problems that are difficult to diagnose, particularly with branch prediction enabled.

2. PowerPC 26-Bit Address Offset Branching

By default, VxWorks uses **bl** or **bla** instructions for exception and interrupt handling, and for dynamically downloaded module relocations. Using **bl** or **bla**, the PowerPC architecture is only capable of branching within the limits imposed by a signed 26-bit offset.

This limits the available branch range to +/- 32 MB from the branch instruction (**bl**) or from address zero (**bla**).

Branching Across Large Address Ranges

Branches across larger address ranges must be made to an absolute address with the help of the LR or CTR register. Each absolute jump is accomplished with a sequence of at least three instructions (more, if the register state must be preserved). This is rarely needed and is expensive in terms of execution speed and code size. Such large branches are typically seen only in large downloaded modules and large (greater than 32 MB) system images.

To get around this restriction for downloadable applications, use the -mlongcall compiler option in the GNU compiler. However, this option can introduce an unacceptable amount of performance penalty and extra code size for some applications. For this reason, the VxWorks kernel is not compiled using -mlongcall.

Note: The DKM loader replaces a PC-relative branch (**bl**) with a zero-relative branch (**bla**) if the branch target is within range of a **bla** but not within range of a **bl**. Because the kernel is usually within 32 MB of address zero, this substitution should succeed in most cases (provided all external function references within a DKM are to functions defined in the kernel) and many DKMs no longer need to be built using -mlongcall.

Extended-Call Exception Vector Support

VxWorks for PowerPC provides support for extended-call (32-bit addressable) exception vectors. To enable support for extended-call exception vectors in your VxWorks image project (VIP), include the INCLUDE_EXC_EXTENDED_VECTORS component.

When exceptions and interrupts occur, PowerPC processors transfer control to a predetermined address, the exception vector, depending on the exception type. After saving volatile task state, the handler function installed for that exception vector is called. This call is made using **bl** or **bla** instructions that, as described previously, require the handler function to be located within the 32 MB of the vector table or within the first 32 MB of memory. Most systems are able to satisfy this 32 MB constraint.

VxWorks provides support for extended-call exception vectors, which can call handler functions located anywhere in the 4 GB address space. Extended-call exception vectors make calls to a 32-bit address in the link register (LR) using the **blrl** instruction. Extra work is required for an extended-call exception vector to load a 32-bit address into the LR, and make a call to it. Therefore, using extended-call exception vectors incurs an additional eleven instruction overhead in increased interrupt latency. Therefore, Wind River recommends that you do not use this feature unless absolutely necessary.

This release of VxWorks still maintains the earlier style 26-bit call vectors as the default. Using a single **bl** or **bla** instruction is much more efficient than the multiple-instruction sequence described previously. Wind River expects that most targets will continue to use the original relative branch (default) style exception handling.

The global boolean, **excExtendedVectors**, allows users to enable extended-call exception vectors. By default, **excExtendedVectors** is set to **FALSE**. When set to **TRUE**, extended-call vectors are enabled. **excExtendedVectors** must be set to **TRUE** before the exception vectors are initialized in the VxWorks boot sequence (that is, before the call to excVecInit( )). Setting **excExtendedVectors** after excVecInit( ) does not achieve the desired result, and results in unpredictable system behavior.

13. POWERPC EXCEPTION AND INTERRUPT CONSIDERATIONS

Depending on your target processor type, there are certain considerations related to exception and interrupt handling that you must be aware of during your development.

Interrupt Vector Table Base Address

On PowerPC Book-E processors, the exception vector prefix register (EVPR) and the interrupt vector prefix register (IVPR), respectively, determine the base address of the interrupt vector table. Only address 0 is supported.

Support on PPCE500V2, PPCE500MC, and PPCE6500 Processors

PPCE500V2, PPCE500MC, and PPCE6500 processors support two classes of exceptions and interrupts: *normal* and *machine check*. Besides the standard excConnect( ) and excIntConnect( ) functions, excCrtConnect( ) and excIntCrtConnect( ) are available for the critical exception class, and excMchkConnect( ) is available for the machine check exception class (see excVecGet( ) and excVecSet( ) below). The function prototypes are the same for all connect functions.

The following table shows the interrupt vector offset registers (IVORs):

Table 1. Interrupt Vector Offset Register Settings for PPCE500V2, PPCE500MC, and PPCE6500

IVOR	Interrupt Type	Offset
IVOR0	Critical input	0x100
IVOR1	Machine check (see note)	0x200
IVOR2	Data storage	0x300
IVOR3	Instruction storage	0x400
IVOR4	External input	0x500
IVOR5	Alignment	0x600
IVOR6	Program	0x700
IVOR7	Floating-point unavailable (not supported on MPC85XX excluding PPCE500MC and PPCE6500)	0x800
IVOR8	System call	0x900
IVOR9	Auxiliary processor unavailable (not supported on MPC85XX excluding PPCE500MC, e5500 and PPCE6500)	0xa00
IVOR10	Decrementer	0xb00

IVOR	Interrupt Type	Offset
IVOR11	Fixed-interval timer interrupt	0xc00
IVOR12	Watchdog timer interrupt	0xd00
IVOR13	Data TLB error	0xe00
IVOR14	Instruction TLB error	0xf00
IVOR15	Debug	0x1000
IVOR32	SPE APU unavailable (not supported for PPCE500MC, e5500, or PPCE6500 AltiVec unavailable (supported only for PPCE6500)	0x1100
IVOR33	SPE floating-point data exception (not supported for PPCE500MC, e5500, or PPCE6500) AltiVec assist (supported only for PPCE6500)	0x1200
IVOR34	SPE floating-point round exception (not supported for PPCE500MC, e5500, or PPCE6500)	0x1300
IVOR35	Performance monitor	0x1400

Note: If cache parity recovery is enabled, IVOR1 is modified to address 0x1500, where the parity recovery code resides. Exception processing falls back to address 0x200 after examining the MCSR if the machine check is not caused by parity error.

Support on PowerPC 603 Processors

PPC603 processors support two classes of exceptions and interrupts: *normal* and *machine check*. The standard excConnect( ) and excIntConnect( ) functions are available for normal exception classes. The function prototypes are the same as that of all connect functions. The PPC603 vector offset is fixed. For more information, see the *MPC603e RISC Microprocessor User's Manual*.

excVecGet( ) and excVecSet( )

In a standard VxWorks image, excVecInit( ) and excInit( ) install the default exception and interrupt handlers, along with the stub for the entry and exit code, by calling the connect functions described previously. Application code can change the default handler to an alternate handler by calling excVecSet( ). excVecSet( ) does not copy the stub for the entry and exit code, and thus, the exception type (normal, critical, or machine check) need not be specified. The default exception type for the vector of interest is used. If the application code changes the location of a vector (for example, using IVOR, which is not recommended), the connect functions are still needed to install the stub as well as the handler. excVecSet( ) is used to install an alternate handler, and excVecGet( ) returns the address of the installed handler given a vector. For example:

```
void excVecSet (FUNCPTR *vectr, FUNCPTR function);
FUNCPTR excVecGet (FUNCPTR *vectr);
```

14. POWERPC SHARED MEMORY CONSIDERATIONS

Multiprocessing systems can encounter problems with updating and sharing memory across CPUs. You must consider these issues during your development.

One issue is that individual CPUs can reorder both read and write operations in order to improve overall system efficiency. From the perspective of a single CPU in the system, this reordering is completely transparent because the CPU ensures that any read operation gets data that was previously written, regardless of the order in which the read and write operations are actually committed to system memory. The reordering occurs in the background, and is never visible to the programmer.

Because of this reordering, two tasks that share data can never assume that the order in which an operation is performed on one CPU is the same as the order in which the operations are written to or read from memory. To enforce an order, if required, use memory barriers to guarantee the ordering of operations between cooperating CPUs.

A second issue in multiprocessing systems is guaranteeing mutual exclusion to a memory location.

Note: For more information on shared memory, see the *VxWorks 7 Platform Programmer's Guide: VxWorks 7 Memory Model*.

15. SIMULTANEOUS MULTITHREADING (SMT) ON PPCE6500

SMT support can be configured in your VxWorks kernel as part of your VxWorks source build (VSB) and image project (VIP) creation for PPCE6500 targets.

Thread support on the PPCE6500 processors enables two logical CPUs per physical core. Each thread acts as a unique CPU with its own register set and execution unit. This means that the two threads on a physical core can simultaneously execute two VxWorks tasks. However, some CPU resources are shared by the two threads. As a result, one thread can impact the execution performance and determinism of its sibling thread.

Thread support for a VxWorks SMP kernel configuration is enabled by default when the VSB is configured for the PPCE6500 architecture. You can disable it again from the VSB configuration tool by turning off SMP_SMT. The number of CPUs configured and enabled for an SMP image is automatically adjusted at VIP creation time depending on the configuration of the VSB. For example, for a system with 12 physical cores with SMP_SMT support enabled in the VSB, the value of VX_SMP_NUM_CPUS in the VIP is 24. Without SMT support enabled in the VSB, it is 12 (one thread per physical core).

The VxWorks SMP scheduler optimization for SMT systems is enabled in the VSB with the SMP_SCHED_SMT_POLICY option. This is included by default. This optimization provides load balancing across physical cores so that, whenever possible, only one of the sibling threads executes a task, while the other remains idle. This is only possible when the system is not fully loaded (that is, there are fewer ready tasks than the number of enabled threads).

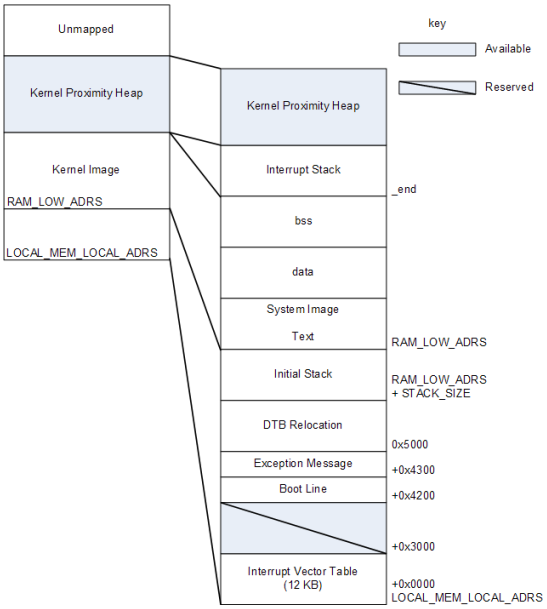
When SMP_SCHED_SMT_POLICY is enabled in the VSB, INCLUDE_SMP_SCHED_SMT_POLICY will be included by default in the VIP.

To switch between the SMT scheduling policy, and the non-SMT optimised scheduling policy, modify the VSB SMP_SMT option, and create a new VIP. For additional information, see the *VxWorks 7 Application Programmer's Guide: SMP Scheduler Policy for Simultaneous Multithreading*.

16. VXWORKS MEMORY LAYOUT FOR POWERPC

VxWorks for PowerPC uses the standard VxWorks memory layout. During your development, you must be aware of the PowerPC-specific offsets used in the kernel system virtual memory region.

The standard VxWorks memory layout is described in the *VxWorks 7 Platform Programmer's Guide: VxWorks 7 Memory Model*. The following figure shows the breakdown of the kernel system virtual memory region for PowerPC processors:



The labels in the figure are defined as follows:

Interrupt Vector Table

Table of exception/interrupt vectors.

Boot Line

ASCII string of boot parameters.

Exception Message

ASCII string of the fatal exception message.

Device Tree Binary (DTB) Relocation

Copied DTB data.

Initial Stack

Initial stack for usrInit( ), until usrRoot( ) is allocated a stack.

System Image

The VxWorks system image itself (three sections: text, data, and bss). The entry point for VxWorks is at the start of this region, which is BSP dependent (see the BSP-specific documentation).

Interrupt Stack

Size is defined by ISR_STACK_SIZE (see INCLUDE_KERNEL). Location depends on the system image size. For VxWorks SMP, this block contains the idle-task stacks and task control blocks (TCBs) and is replicated for each CPU.

Kernel Proximity Heap

Size depends on the size of the system image. **memPoolEndAdrs** returns the address of the end of the free memory pool.

All addresses shown in the figure above are relative to the start of memory for a particular target board.

The above figure shows the interrupt vector table starting at address 0. For more information on the placement of interrupt and exception vectors, see PowerPC Exception and Interrupt Considerations on page 47.